NAME: Manoj Kumar
SRN: PES1UG24CS264

Dynamic Host Blocking using SDN (Mininet + POX)

---

1. Objective

The objective of this project is to design and implement a Software Defined Networking (SDN) solution that dynamically monitors network traffic and blocks hosts exhibiting abnormal behavior. The system detects high traffic from a host and prevents further communication.

---

2. Introduction

Software Defined Networking (SDN) separates the control plane from the data plane, allowing centralized control of network behavior.

In this project, Mininet is used to simulate a network and POX is used as the controller. The controller monitors packet flow and applies rules dynamically. If a host sends excessive traffic, it is identified as suspicious and blocked.

6. Results---

3. Tools Used

• Mininet (Network Emulator)
• POX Controller (OpenFlow Controller)
• Python

---

4. Setup and Execution

Step 1: Start POX Controller
```
cd ~/pox
python3 pox.py misc.traffic_block
```

Step 2: Start Mininet (Open a new terminal)
```
sudo mn --controller=remote
```

Step 3: Test Network
pingall

Step 4: Generate Traffic
h1 ping h2

---

5. Working Principle

The controller listens for incoming packets and maintains a count of packets sent by each host.

If traffic is within normal limits, packets are forwarded normally.
If traffic exceeds the threshold, the host is identified as suspicious and blocked.
Blocking is done by dropping packets dynamically.

This demonstrates real-time traffic monitoring and control.

6.1 Normal Communication

```
idrk@ubuntu: ~/sdn-project
idrk@ubuntu:~/pox$ sudo mn --controller=remote
[sudo] password for idrk:
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6633
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

All hosts communicate successfully with 0% packet loss.

6.2 During High Traffic

```
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.30 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.35 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=2.08 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=1.52 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=1.32 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=1.40 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=1.47 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=1.19 ms
^C
--- 10.0.0.2 ping statistics ---
17 packets transmitted, 8 received, 52.9412% packet loss, time 16723ms
rtt min/avg/max/mdev = 1.194/1.453/2.080/0.254 ms
```

Packet loss increases, indicating that blocking has started.

6.3 Controller Logs

```
idrk@ubuntu:~/pox/pox/misc$ cd ~/pox
python3 pox.py misc.traffic_block
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:POX requires one of the following versions of Python:
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported versio
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:misc.traffic_block:Host 10.0.0.1 count = 1
INFO:misc.traffic_block:Host 10.0.0.2 count = 1
INFO:misc.traffic_block:Host 10.0.0.2 count = 2
INFO:misc.traffic_block:Host 10.0.0.1 count = 2
INFO:misc.traffic_block:Host 10.0.0.1 count = 3
INFO:misc.traffic_block:Host 10.0.0.2 count = 3
INFO:misc.traffic_block:Host 10.0.0.1 count = 4
INFO:misc.traffic_block:Host 10.0.0.2 count = 4
INFO:misc.traffic_block:Host 10.0.0.1 count = 5
INFO:misc.traffic_block:Host 10.0.0.2 count = 5
INFO:misc.traffic_block:Host 10.0.0.1 count = 6
INFO:misc.traffic_block:Host 10.0.0.2 count = 6
INFO:misc.traffic_block:Host 10.0.0.1 count = 7
INFO:misc.traffic_block:Host 10.0.0.2 count = 7
INFO:misc.traffic_block:Host 10.0.0.1 count = 8
INFO:misc.traffic_block:Host 10.0.0.2 count = 8
INFO:misc.traffic_block:Host 10.0.0.1 count = 9
INFO:misc.traffic_block:Host 10.0.0.2 count = 9
INFO:misc.traffic_block:Host 10.0.0.1 count = 10
INFO:misc.traffic_block:Host 10.0.0.2 count = 10
INFO:misc.traffic_block:Host 10.0.0.1 count = 11
INFO:misc.traffic_block:BLOCKING 10.0.0.1 due to high traffic
```

The controller detects high traffic and displays:
BLOCKING 10.0.0.X

---

7. Conclusion

This project successfully demonstrates dynamic traffic monitoring and host blocking using SDN principles. The controller efficiently detects abnormal behavior and prevents network misuse by blocking suspicious hosts.

---

8. References

https://mininet.org/
https://github.com/noxrepo/pox